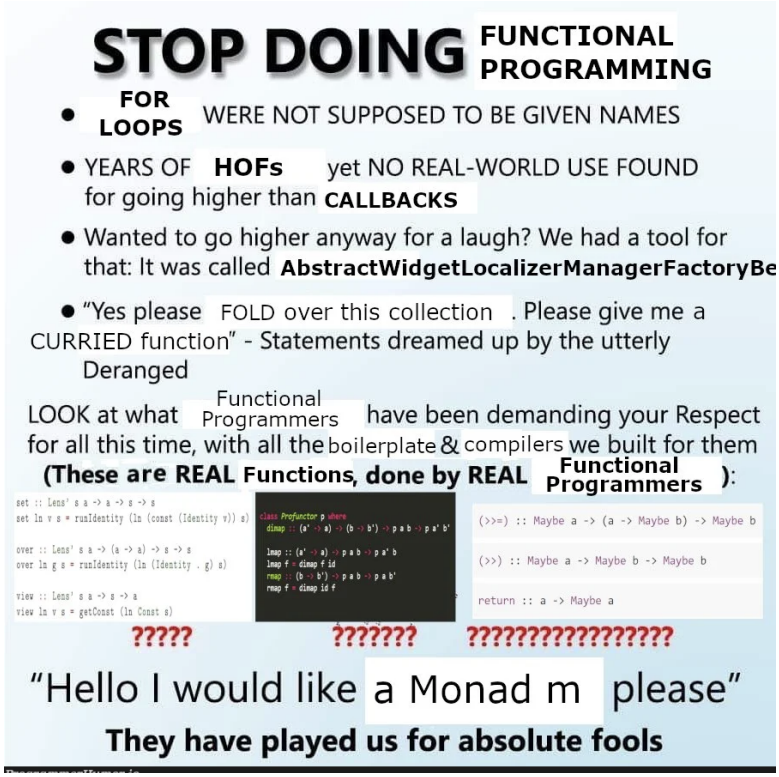
Sapienza Università di Roma

Tecniche di Programmazione Funzionale e Imperativa

libro del corso:
slide del professor Salvo



aglaia norza

15 marzo 2026

thisisaglaia@gmail.com
github.com/AglaiaNorza

Indice

1 Le basi **2**

1.1 Concetti fondamentali 2

1.2 Sintassi, Funzioni e Pattern Matching 3

1.2.1 Definizione e Applicazione 3

1.2.2 Lambda Espressioni e Combinatori 3

1.2.3 Pattern matching 3

1.2.4 Scope locale: `let` e `where` 4

1.3 Sistema dei Tipi e Classi 4

1.3.1 Type Signatures 4

1.3.2 Type Classes 5

1.3.3 Definizione di Tipi: `type` e `data` 6

1.3.4 Istanze e Derivazione (`deriving`) 6

1.4 Strutture Dati Predefinite 7

1.4.1 Tipi di Base e Tuple 7

1.4.2 Liste 7

1.5 Funzionali di Ordine Superiore 8

1.5.1 Famiglia dei `fold` (schemi di ricorsione) 9

1

Le basi

1.1 Concetti fondamentali

- **Valutazione lazy:** Haskell valuta le espressioni solo quando è strettamente necessario (permette di gestire agevolmente strutture dati infinite)
- **Trasparenza referenziale:** non essendoci assegnazioni o memoria mutabile (assenza di side-effects), una variabile identifica sempre lo stesso oggetto (permette di manipolare algebricamente i programmi)
- **Polimorfismo e type inference:** molte funzioni operano su tipi generici (es. `a`, `b`). Il compilatore inferisce automaticamente il *tipo principale* (il più generale possibile); tutti gli altri tipi corretti sono sue istanze.
- **Non-terminazione, Bottom e `undefined`:** in Haskell, ogni tipo possiede implicitamente un valore speciale (chiamato *bottom*, indicato matematicamente con $\perp$) che rappresenta un'eccezione fatale o una computazione che entra in un loop infinito
  - **la costante `undefined`:** è la materializzazione a livello di codice del valore $\perp$ - poiché una computazione che fallisce o non termina può avvenire in qualsiasi contesto (mentre si aspetta un numero, un booleano, ecc.), `undefined` possiede il tipo più generale e polimorfo possibile: `a` (ovvero $\forall\alpha.\alpha$)
  - **Il limite del lambda calcolo (*Occurs check*):** nel lambda calcolo puro, il loop infinito per eccellenza è il combinatore $\Omega = (\lambda x.xx)(\lambda x.xx)$. In Haskell, l'auto-applicazione $\lambda x.xx$ **non è tipabile**. Perché x possa prendere se stesso come argomento, il suo tipo α dovrebbe essere uguale a una funzione che prende α come parametro e restituisce β ($\alpha = \alpha \to \beta$). Questo creerebbe un tipo infinito (durante l'inferenza dei tipi, il compilatore esegue un controllo chiamato *Occurs check* che impedisce alla variabile di tipo α di apparire all'interno della sua stessa definizione, rifiutando di conseguenza l'espressione)
  - **Ricorsione esplicita:** poiché il type system ci vieta di creare loop tramite l'auto-applicazione anonima di $\lambda x.xx$, in Haskell la non-terminazione si esprime unicamente attraverso la ricorsione esplicita. Definendo ad esempio `omega' = omega'`, creiamo intenzionalmente una computazione divergente che il compilatore accetta, assegnandole il tipo più generale `a`.
- **Ideologia funzionale:** l'ideologia funzionale favorisce l'assemblaggio di funzioni semplici tramite composizione (`.`). Si pensa in termini di trasformazioni globali di strutture dati, per poi raffinare in un secondo momento i colli di bottiglia prestazionali.

1.2 Sintassi, Funzioni e Pattern Matching

1.2.1 Definizione e Applicazione

- l'applicazione di funzione: **associa a sinistra** e si scrive senza virgole (`f x y`).
- **aperatori infissi vs prefissi:** le funzioni binarie si usano in modo infisso tra backtick (`10 ‘div‘ 2`), gli operatori infissi in modo prefisso tra parentesi (`(+) 3 4`).

1.2.2 Lambda Espressioni e Combinatori

Le funzioni anonime si scrivono con backslash (`\` simula λ).

```
-- identita' (combinatore I)
i :: t -> t
i x = x           -- definizione standard
i' = \x -> x      -- lambda espressione

-- combinatore K (proiettore del primo argomento) (primo assioma di Hilbert)
k :: t1 -> t2 -> t1
k' = \x y -> x

-- combinatore S (sostituzione) (secondo assioma di Hilbert)
s :: (t2 -> t1 -> t) -> (t2 -> t1) -> t2 -> t
s x y z = x z (y z)
```

1.2.3 Pattern matching

Il pattern matching decompone le strutture dati in base alla loro forma sintattica. L'ordine delle clausole è strettamente sequenziale (viene scelta la prima dall'alto che fa matching).

- **mancanza di unificazione:** non si possono usare variabili identiche nel pattern per verificarne l'uguaglianza (es. `f x x = ...` è invalido)
- **underscore (`_`):** variabile anonima ("don't care") per valori non rilevanti
- **as-pattern (`@`):** permette di dare un nome all'intera struttura mentre la si decompone (es. `xs@(x:txs)`)
- **guardie (`|`):** permettono di definire il comportamento di una funzione valutando sequenzialmente una serie di *predicati booleani* (condizioni logiche); a differenza del pattern matching che controlla la *forma* sintattica dei dati, le guardie valutano *valori e relazioni* (es. `x > 0`, `x == y`)
  - l'ultima guardia è tipicamente `otherwise` (che nel Prelude di Haskell è semplicemente un alias per `True`), che funge da caso di default ("catch-all") per assicurarsi che la funzione restituisca sempre un valore.

```
confronta x y
  | x == y    = "i numeri sono uguali" -- supera la mancanza di unificazione
  | x > y     = "il primo e' maggiore"
  | otherwise = "il secondo e' maggiore"
```

- **equazioni multiple (definizione per casi):** il modo più comune di fare pattern matching è definire la funzione scrivendo più "intestazioni" separate; il compilatore le prova dall'alto verso

il basso: se gli argomenti combaciano con la forma (pattern) della riga, esegue quell'equazione, altrimenti controlla la successiva.

```
saluta "mattina" = "buongiorno"
saluta "sera"    = "buonasera"
saluta _         = "ciao"
```

1.2.4 Scope locale: `let` e `where`

- **`let ... in`**: costrutto standalone per definire variabili/funzioni locali (utile per strutturare/destrutturare tuple nelle ricorsioni)
- **`where`**: clausola che qualifica la parte destra di una definizione appena conclusa

```
fibAux n = let (f, fPrec) = fibAux (n-1) in (f + fPrec, f)
-- equivalente a:
fibAux' n = (f + fPrec, f) where (f, fPrec) = fibAux' (n-1)
```

> differenza tra `let` e `where`
>
> una clausola `where` è visibile da tutte le guardie di una specifica equazione. Un `let` definito dopo il simbolo `=` di una guardia è visibile solo all'interno di quella singola guardia. Per condividere un calcolo tra più guardie, il `where` è obbligatorio.

```
-- condivisione tra guardie
analizzaRaggio r
  | area > 100 = "cerchio grande"
  | area < 10  = "cerchio piccolo"
  | otherwise  = "cerchio medio"
  where area = pi * r^2  -- 'area' calcolata una volta e visibile a tutti i pipe

-- inline: (serve il let perche' non c'e' una dichiarazione di funzione)
volumeCilindro r h = (let area = pi * r^2 in area) * h
```

1.3 Sistema dei Tipi e Classi

1.3.1 Type Signatures

In Haskell, grazie al meccanismo di type inference, il compilatore è in grado di dedurre automaticamente il tipo più generale per quasi ogni espressione. Tuttavia, è considerata un'ottima pratica (e in alcuni casi più complessi è strettamente necessario) dichiarare esplicitamente il tipo di una funzione subito sopra la sua definizione.

Questo si fa usando l'operatore `::` (che si legge "ha tipo"). Dichiarare esplicitamente i tipi è utile ai fini della leggibilità del codice e permette di "forzare" il tipo desiderato, aiutando il compilatore a individuare eventuali errori logici in modo molto più preciso.

```
-- dichiarazione esplicita del tipo: prende due Int e restituisce un Int
somma :: Int -> Int -> Int
somma x y = x + y
```

1.3.2 Type Classes

Le classi limitano il polimorfismo imponendo vincoli (o "interfacce") sui tipi. Più che semplici insiemi, rappresentano classificazioni di tipo algebrico: raggruppano i tipi su cui sono definite certe operazioni. Le classi possono formare gerarchie, dove una sottoclasse "estende" la superclasse ereditandone le operazioni e aggiungendone di nuove.

- **Eq (Equality Types):** comprende i tipi che ammettono l'uguaglianza (`==`) e la sua negazione (`/=`). Tutti i tipi base (Bool, Int, Float, ecc.) sono istanze di Eq, così come le liste e tuple costruite su tipi Eq

  definizione interna di Eq

```
class Eq a where
  (==), (/=) :: a -> a -> Bool
  -- le classi possono contenere codice (metodi di default)
  x /= y = not (x == y)
```

- **Ord (Ordered Types):** sottoclasse di `Eq` - tipi ordinabili che supportano operazioni come `<=` , `<` , `>=` , `>` , `min` e `max` .
  - per definire un'istanza di Ord, è sufficiente fornire il codice per l'operatore `<` , il resto può essere interdefinito.
- **Num:** la classe più generica per i tipi numerici. È sottoclasse di `Eq` e `Show` . Implementa operazioni aritmetiche base (`+` , `-` , `*`), oltre a valore assoluto (`abs`) e segno (`signum`).

  i numeri interi costanti sono polimorfi rispetto alla classe `Num` :

```
> :t 3
3 :: Num a => a  -- 3 puo' essere un Int, Integer, Float, ma non un Char
```

- **Altre classi utili:**
  - **Show:** tipi che possono essere stampati (convertiti in `String`) tramite il metodo `show` . Se provate a stampare a video il risultato di una funzione che non appartiene a `Show` (es. una lambda anonima come `\x -> x`), l'interprete darà errore.
  - **Read:** permette di trasformare una `String` in un valore tramite il metodo `read`
  - **Integral:** sottoclasse di `Num` per i numeri interi (es. Int, Integer); aggiunge le operazioni di divisione intera `div` e modulo `mod`
  - **Fractional:** sottoclasse di `Num` per i numeri frazionari (es. Float, Rational); aggiunge l'operatore di divisione reale `(/)` e `recip`

1.3.2.1 Vincoli di classe

I vincoli di classe si indicano prima del tipo vero e proprio, separati dalla doppia freccia `=>` . Vincoli multipli si racchiudono tra parentesi e indicano che la funzione opera su un tipo che deve soddisfare tutte le classi elencate.

```
-- esempio: mcd richiede che il tipo 'a' supporti sia l'ordine (<, ==)
-- sia le operazioni aritmetiche (-)
mcd :: (Ord a, Num a) => a -> a -> a
mcd x y
```

```
  | x == y    = x
  | x < y     = mcd (y - x) x
  | otherwise = mcd (x - y) y
```

1.3.3 Definizione di Tipi: `type` e `data`

- **`type` (sinonimi):** danno nuovi nomi a tipi esistenti (es. `type Casella = (Int, Int)`)
- **`data` (tipi algebrici):** costruiscono nuovi tipi tramite costruttori finiti, parametrici o ricorsivi. Possono essere dichiarati istanze di classi.

```haskell
data SetteNani = Eolo | Pisolo | Brontolo          -- finiti
data Either a b = Left a | Right b                 -- unioni disgiunte
data Maybe a = Just a | Nothing                    -- gestione fallimenti
data List a = Nil | Cons a (List a)                -- ricorsivi/induttivi
data BTree a = Foglia a | Radice (BTree a) (BTree a)
```

1.3.3.1 Il costruttore di tipo `Maybe`

Il tipo `Maybe` è il tipo di computazioni che possono fallire. In Haskell, è il tipo con cui si rappresentano comunemente le eccezioni in maniera sicura e controllata dal sistema di tipi (corrisponde a `optional`)

Un valore di tipo `Maybe` può assumere due forme:

- `Just v`: contiene un valore `v` (la computazione ha avuto successo e restituisce qualcosa)
- `Nothing`: non contiene nulla (rappresenta una computazione indefinita o fallita)

Si può utilizzare per le *funzioni parziali* (che non sono definite per tutti i possibili valori di input (eg divisione per zero)). Tramite il pattern matching, possiamo "spacchettare" il dato per estrarne il contenuto.

```haskell
-- definizione del tipo predefinito
data Maybe a = Just a | Nothing

-- 1. costruzione di un Maybe: divisione sicura
-- Se il divisore e' 0 restituisce Nothing, altrimenti restituisce Just il
    risultato
safediv :: Integral a => a -> a -> Maybe a
safediv n 0 = Nothing
safediv n m = Just (n `div` m)

-- 2. destrutturare un Maybe: funzione "inversa" per estrarre il valore
extract :: Maybe a -> a
extract (Just n) = n
extract Nothing  = undefined

-- extract (safediv 30 0) -> *** Exception: Prelude.undefined
```

1.3.4 Istanze e Derivazione (`deriving`)

La keyword `deriving` è un meccanismo automatico che permette al compilatore di generare le istanze per le classi predefinite senza dover scrivere codice manuale.

- **Eq:** genera un'uguaglianza basata sulla sintassi
- **Ord:** deriva l'ordine lessicografico in base alla sequenza dei costruttori
- **Show/Read:** la conversione usa i nomi dei costruttori come stringhe
- **Enum:** permette di generare liste per enumerazione (es. `[Lun..Ven]`)

```haskell
-- istanze automatiche tramite deriving
data Giorni = Lun | Mar | Mer | Gio | Ven | Sab | Dom
    deriving (Eq, Ord, Show, Enum, Read)

-- esempio di istanza manuale
instance Eq Bool where
    False == False = True
    True == True   = True
    _ == _         = False
```

1.4 Strutture Dati Predefinite

1.4.1 Tipi di Base e Tuple

- **Booleani:** `Bool` (`True`, `False`). Operatori: `not`, `&&`, `||`.
- **Interi:** `Int` (dimensione fissa), `Integer` (precisione illimitata).
- **Tuple:** n-uple fisse e disomogenee (es. `(String, Bool)`)

  offrono le funzioni *su coppie*: `fst`, `snd` (prendono primo e secondo argomento)

1.4.2 Liste

Sequenze di elementi omogenei. Definite dal costruttore vuoto `[]` e dall'operatore infisso *cons* `(:)`. Le stringhe sono `[Char]`.

Zucchero sintattico: `[1,2,3]` equivale a `1:2:3:[]`.

```haskell
-- enumerazioni
[m..n]           -- range finito (lista con elementi da m a n)
[m, n..p]        -- range con step (es. [1, 3..11] dispari)
[m..]            -- lista infinita (lazy)
```

Haskell supporta le **list comprehension** (definizioni set-like del contenuto di una lista)

```haskell
-- list comprehension (spesso tradotte internamente in map/filter)
[x^2 | x <- [1..5]]                -- quadrati
[x | x <- xs, x `mod` 2 == 0]      -- filtri / guardie
[(x,y) | x <- xs, y <- ys]         -- prodotto cartesiano
```

1.4.2.1 Funzioni su Liste

- **Estrazione:** `head`, `tail`. `last`, `init`. `(!!)` per estrarre all'indice n

```haskell
head [1, 2, 3]  -- risultato: 1
tail [1, 2, 3]  -- risultato: [2, 3]
last [1, 2, 3]  -- risultato: 3
init [1, 2, 3]  -- risultato: [1, 2]
[1, 2, 3] !! 1  -- risultato: 2
```

- **Ispezione:** `null`, `length`

```haskell
null []          -- risultato: True
length [1, 2, 3] -- risultato: 3
```

- **Taglio:** `take n`, `drop n`, `splitAt n` (tupla con take e drop).

```haskell
take 2 [1, 2, 3, 4]    -- risultato: [1, 2]
drop 2 [1, 2, 3, 4]    -- risultato: [3, 4]
splitAt 2 [1, 2, 3, 4] -- risultato: ([1, 2], [3, 4])
```

- **Generali:** `++` (concatenazione), `reverse`

```haskell
[1, 2] ++ [3, 4]  -- risultato: [1, 2, 3, 4]
reverse [1, 2, 3] -- risultato: [3, 2, 1]
```

- **Matematica/logica:** `sum`, `minimum`, `maximum`. `and`, `or`. `any p`, `all p` (predicati su almeno uno o tutti)

```haskell
sum [1, 2, 3]       -- risultato: 6
minimum [5, 2, 9]   -- risultato: 2
maximum [5, 2, 9]   -- risultato: 9
and [True, False]   -- risultato: False
or [True, False]    -- risultato: True
any (> 2) [1, 2, 3] -- risultato: True (almeno uno è > 2)
all (> 0) [1, 2, 3] -- risultato: True (tutti sono > 0)
```

- **Liste annidate/multiple:** `concat` (appiattisce liste di liste), `zip` (unisce liste in coppie, fermandosi alla più corta)

```haskell
concat [[1, 2], [3, 4]]    -- risultato: [1, 2, 3, 4]
zip [1, 2] ['a', 'b', 'c'] -- risultato: [(1, 'a'), (2, 'b')]
```

1.5 Funzionali di Ordine Superiore

Funzioni che prendono/restituiscono altre funzioni. Favoriscono la composizionalità.

```haskell
-- composizione (.) : Equivale a f(g(x))
(.) f g x = f (g x)

-- curry e uncurry (Isomorfismo A x B -> C  <==>  A -> (B -> C))
curry :: ((a, b) -> c) -> a -> b -> c
uncurry :: (a -> b -> c) -> (a, b) -> c

-- flip: inverte l'ordine dei parametri
flip :: (a -> b -> c) -> b -> a -> c

-- map e filter
map :: (a -> b) -> [a] -> [b]         -- proprieta': map (f . g) = map f . map g
```

```haskell
filter :: (a -> Bool) -> [a] -> [a]

-- zipWith: applica f. binaria agli elementi di due liste
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
```

1.5.1 Famiglia dei `fold` (schemi di ricorsione)

I `fold` astraggono il concetto di ricorsione strutturale sulle liste, sostituendo i costruttori della struttura dati (l'operatore `(:)` e la lista vuota `[]`) con funzioni e valori specifici. Se l'operatore `#` è associativo e ha `e` come elemento neutro, allora `foldl (#) e` = `foldr (#) e`.

- **`foldr` (right fold):** associa a destra; la riduzione avviene "al ritorno" dalle chiamate ricorsive (costruisce espressioni).
  - sostituisce ricorsivamente il costruttore `(:)` con la funzione `f` data e `[]` con il valore base `z`.
  - espansione: `foldr f z [x1, x2, x3]` diventa `x1 ‘f‘ (x2 ‘f‘ (x3 ‘f‘ z))`
  - può terminare anche su liste infinite se la funzione `f` non è stretta nel suo secondo argomento (ovvero se non ha sempre bisogno di valutare la ricorsione per restituire un risultato).

```haskell
-- somma con foldr
foldr (+) 0 [1, 2, 3]
-- valutazione: 1 + (2 + (3 + 0)) = 6

-- implementazione di map usando foldr
myMap f xs = foldr (\x acc -> f x : acc) [] xs
```

- **`foldl` (left fold):** associa a sinistra; simula un ciclo iterativo passando un accumulatore "in avanti"
  - valuta l'espressione partendo da sinistra e procedendo verso l'interno - valuta tutto solo alla fine della lista
  - espansione: `foldl f z [x1, x2, x3]` diventa `((z ‘f‘ x1) ‘f‘ x2) ‘f‘ x3`
  - *nota bene:* In `foldl`, la funzione prende come primo argomento l'accumulatore, e come secondo l'elemento corrente della lista.

```haskell
-- sottrazione con foldl
foldl (-) 0 [1, 2, 3]
-- valutazione: ((0 - 1) - 2) - 3 = -6

-- sottrazione con foldr per confronto
foldr (-) 0 [1, 2, 3]
-- valutazione: 1 - (2 - (3 - 0)) = 2

-- reverse usando foldl
myReverse xs = foldl (\acc x -> x : acc) [] xs
```

- **`foldr1` e `foldl1`:** varianti che non richiedono un valore iniziale esplicito, poiché usano il primo o l'ultimo elemento come base; *danno errore su lista vuota*.

```
-- utile quando non esiste un elemento neutro sensato (es. min/max assoluto)
myMinimum = foldr1 min
myMinimum [5, 3, 8] -- risultato: 3
-- myMinimum []     -- genera errore a runtime
```